Universidad de El Salvador
Facultad Multidisciplinaria de Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software | Modalidad En Línea
Asignatura: Introducción al Software Libre - CICLO II-2026

Proyecto: Servidor Linux con Docker

Grupo N° 2 - Informe Técnico

Objetivo General del Proyecto: Aplicar los fundamentos operativos y de administración de sistemas basados en Software Libre para diseñar, configurar e implementar un servidor Linux automatizado. Este despliegue se apoya en la tecnología de contenedores Docker para garantizar la portabilidad y el aislamiento de servicios, integrando además buenas prácticas de ciberseguridad, gestión estricta de permisos, monitoreo mediante tareas programadas y control de versiones colaborativo.

Repositorio Oficial (Git): https://github.com/Gabriel-UES/proyecto-linux-grupo2.git

Equipo de Trabajo y Roles

Nombre del Estudiante	Rol Asignado y Responsabilidades
Gabriel Antonio Call Ortiz	Infraestructura Base, Gestión de Permisos Avanzados (SGID) y Control de Versiones
Ronald José Bollates Escamilla	Automatización (Cron), Scripts de Monitoreo (Bash) y Despliegue de Docker Básico
Manuel Alexander Zepeda Grijalva	Infraestructura Docker Avanzada, Redes de Contenedores y Configuración en AWS EC2

1. Preparación del Entorno Servidor y Control de Versiones

El proyecto dio inicio con la configuración a nivel de sistema operativo (host). Para establecer una identidad clara en la red, se configuró el hostname de la máquina como 'servidor-grupo2'. A continuación, se implementó un modelo de seguridad basado en el principio de mínimo privilegio. Para ello, se crearon grupos de trabajo específicos ('soporte' y 'web') y se aprovisionaron tres cuentas de usuario distintas:

- **adminsys:** Usuario con privilegios de superusuario (añadido al grupo 'sudo') encargado de la administración general.
- **tecnico:** Asignado al grupo 'soporte', responsable del mantenimiento operativo.
- **visitante:** Asignado al grupo 'web', con permisos limitados únicamente para la visualización y gestión superficial de recursos web.

Para garantizar un entorno de trabajo colaborativo seguro, se diseñó la estructura de directorios bajo la ruta absoluta '/proyecto/', dividiéndola en subcarpetas lógicas: 'datos', 'web', 'scripts' y 'capturas'. Un aspecto técnico crucial fue la aplicación del permiso especial SGID (Set Group ID) mediante el comando 'chmod g+s' en los directorios 'datos' y 'web'. Esta configuración asegura que cualquier archivo creado dentro de estos directorios herede automáticamente el grupo propietario del directorio padre, evitando conflictos de permisos entre los usuarios del mismo equipo.

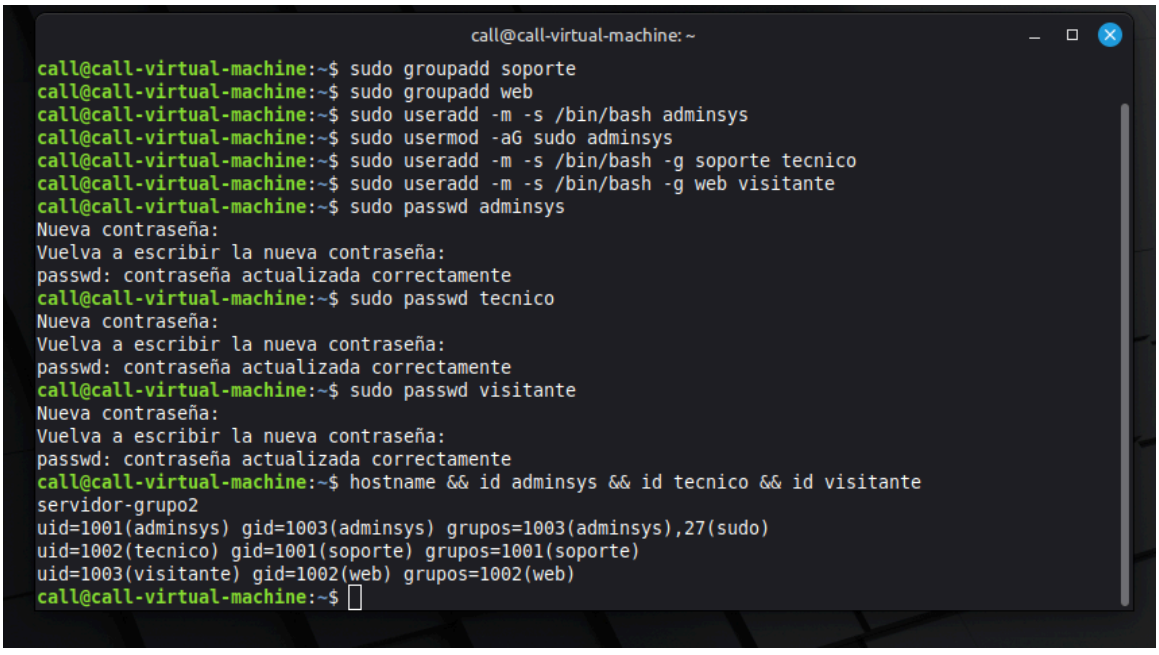


Figura: Creación de usuarios, grupos y verificación de IDs.

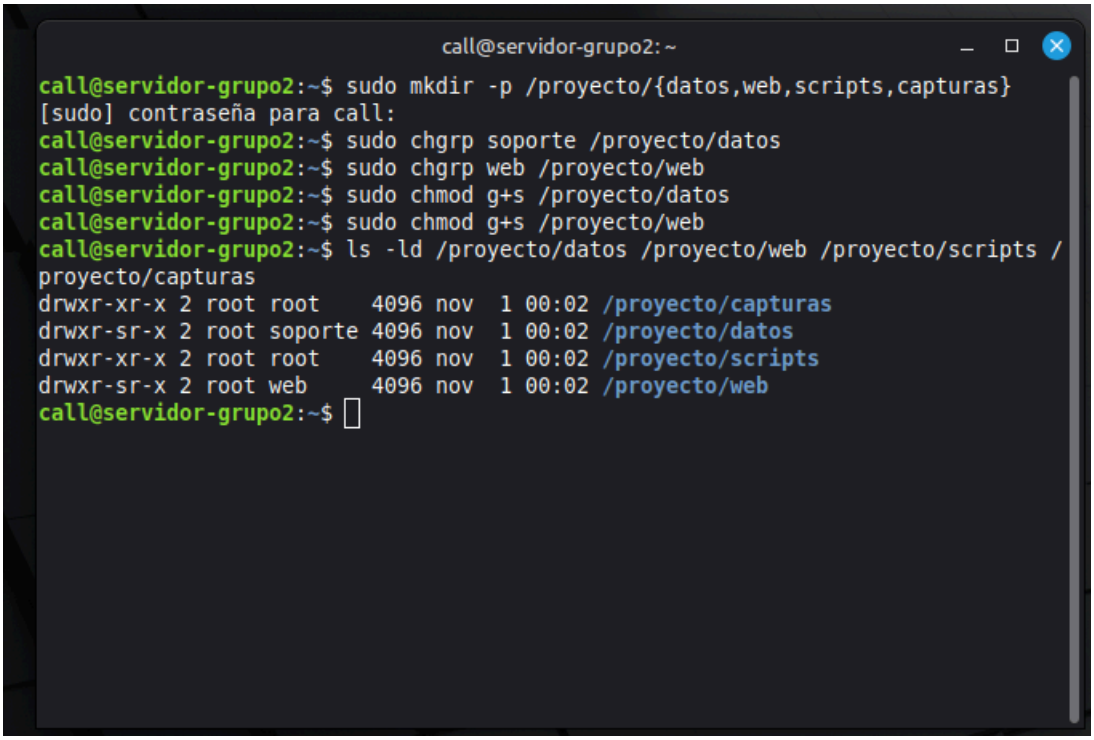


Figura: Configuración de directorios y comprobación de permisos SGID (bandera 's').

Finalmente, la estructura de directorios y los archivos iniciales de configuración fueron integrados a un sistema de control de versiones utilizando Git. Se realizó el 'commit' inicial y se impulsaron los cambios (push) a la rama 'main' del repositorio remoto en GitHub, centralizando así el código fuente del equipo.

2. Automatización y Monitoreo Continuo

La observabilidad del servidor es un requisito fundamental para mantener el rendimiento y la disponibilidad. Para abordar esto, se desarrolló un script de administración automatizado en Bash, denominado 'reporte_sistema.sh'. Este script fue programado con comandos nativos de Linux para extraer y formatear métricas críticas en tiempo real, tales como:

- • Marca de tiempo exacta de la ejecución.
- • Lista de usuarios actualmente conectados al sistema.
- • Estado del almacenamiento y espacio libre en los discos (comando df).
- • Consumo de memoria RAM y memoria Swap disponible (comando free).
- • Recuento total de contenedores Docker en ejecución activa.

Para transformar este script en un sistema de monitoreo pasivo, se utilizó el demonio 'cron'. Se editó la tabla de tareas (crontab) para programar la ejecución ininterrumpida del script cada 30 minutos (*/30 * * * *). Toda la salida generada, incluyendo posibles errores (2>&1), se redirigió a un archivo de registro centralizado ubicado en '/var/log/proyecto/reporte_sistema.log', permitiendo auditorías posteriores sin intervención manual.



Figura: Código del script reporte_sistema.sh, configuración de crontab y logs.

3. Instalación de Docker y Servidor Web Containerizado

3.1. Motor Docker y Despliegue de Nginx Básico

La transición hacia una arquitectura de microservicios requirió la instalación de Docker Engine, Containerd y Docker Compose. Estos paquetes se obtuvieron desde los repositorios oficiales y se instalaron mediante el gestor APT. Para facilitar el flujo de trabajo y evitar el uso constante de privilegios de superusuario, los usuarios operativos (como 'tecnico') fueron agregados al grupo 'docker'. El correcto funcionamiento del motor de contenedores se validó descargando y ejecutando la imagen de prueba 'hello-world'.



Figura: Proceso de descarga e instalación de los paquetes de Docker en la terminal.

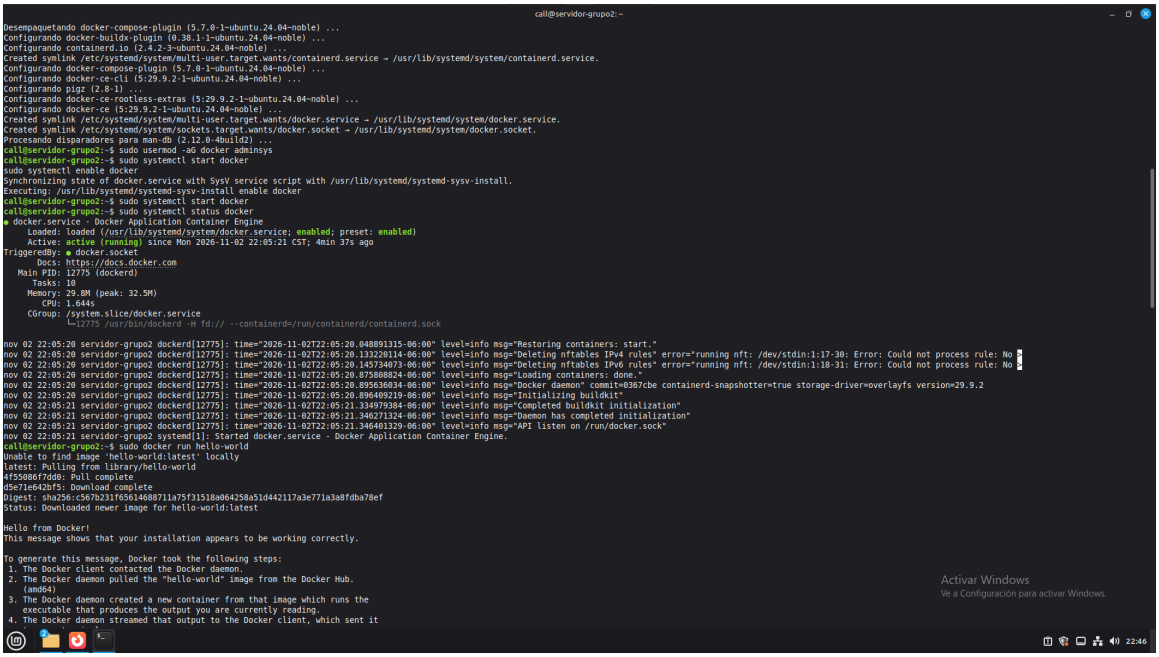


Figura: Ejecución exitosa del contenedor hello-world y verificación del demonio Docker.

Una vez validado el entorno, se procedió a desplegar un servidor web ligero utilizando la imagen oficial de Nginx. El contenedor fue ejecutado en segundo plano (modo

detached) y se configuró un mapeo de puertos (port forwarding) que vinculó el puerto 8080 del servidor anfitrión con el puerto 80 del contenedor. Además, mediante el uso de volúmenes (bind mounts), se enlazó el directorio local '/proyecto/web/' hacia la ruta interna de Nginx, sirviendo de manera exitosa un archivo 'index.html' personalizado creado por el equipo.



Figura: Despliegue del contenedor Nginx y prueba de acceso local (curl localhost:8080), Prueba del servidor corriendo

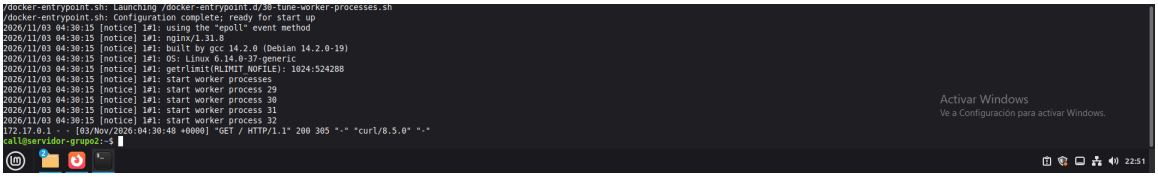


Figura: Revisión de los logs generados por el contenedor Nginx mostrando peticiones HTTP.

3.2. Infraestructura Avanzada y Migración a AWS EC2

Hasta este punto, el servidor Nginx se había desplegado con la imagen oficial y un directorio montado. Para tener un servicio portable y reproducible, que no dependa de archivos del anfitrión, se construyó una imagen propia a partir de un Dockerfile.

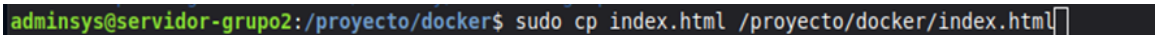
3.2.1 Construcción de una imagen personalizada con Dockerfile.

Se creó el directorio /proyecto/docker, que contiene todo lo necesario para construir la imagen. Se usó el usuario adminsys, que tiene privilegios de superusuario y pertenece al grupo docker.

```
None
mkdir -p /proyecto/docker && cd /proyecto/docker
```

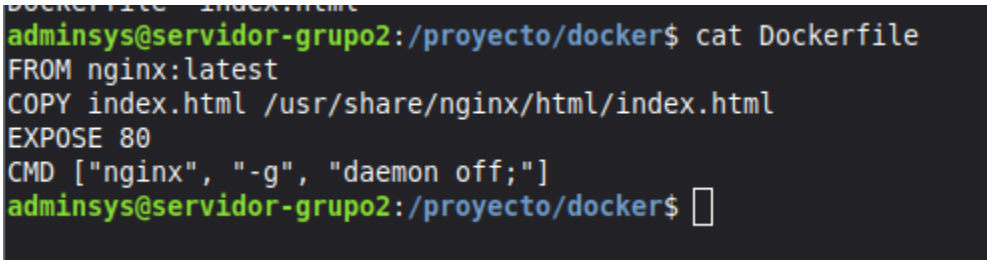
Docker solo puede copiar archivos que estén dentro del contexto de construcción (el directorio donde se ejecuta docker build). Por eso se copió allí el index.html personalizado que está en /proyecto/web/.

```
None
sudo cp /proyecto/web/index.html /proyecto/docker/index.html
```



Definición del Dockerfile. Se creó el archivo con cuatro instrucciones: FROM toma como base la imagen oficial nginx:latest; COPY incorpora la página personalizada al directorio que Nginx publica; EXPOSE 80 documenta el puerto en el que escucha el servicio; y CMD mantiene Nginx en primer plano (daemon off;), requisito para que el contenedor siga activo.

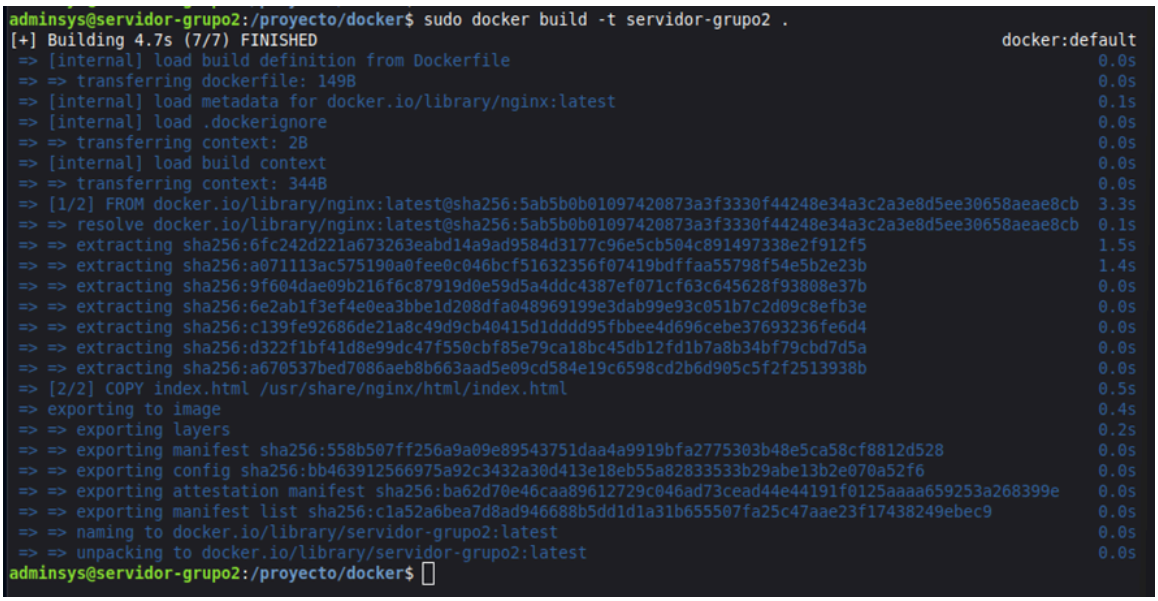
```
None
cat Dockerfile
```



Construcción de la imagen. La opción -t asigna el nombre servidor-grupo2, y el punto final indica que el contexto es el directorio actual. En la salida se aprecian los pasos del Dockerfile ejecutándose en orden.

None

```
sudo docker build -t servidor-grupo2 .
```



Ejecución del contenedor. Se lanzó en segundo plano (-d) con el nombre servidor-dockerfile. La opción -p 8081:80 enlaza el puerto 8081 del anfitrión con el puerto 80 del contenedor.

None

```
sudo docker run -d --name servidor-dockerfile -p 8081:80 servidor-grupo2
```



Pruebas y comprobaciones. Con curl se verificó que el servicio responde con la página personalizada. Con docker images se confirmó que la imagen servidor-grupo2 existe junto a nginx y hello-world, y con docker ps que el contenedor está activo con el mapeo 0.0.0.0:8081->80/tcp.

None

```
curl localhost:8081
```

None

```
sudo docker images
```

None

```
sudo docker ps
```

```
adminsys@servidor-grupo2:/proyecto/docker$ curl localhost:8081
<!DOCTYPE html>
<html>
<head>
    <title>Servidor Grupo 2 - Proyecto ISL</title>
</head>
<body style="font-family: Arial; text-align: center; margin-top: 50px;">
    <h1>¡Bienvenido al Servidor Web Nginx!</h1>
    <p>Proyecto Automatizado con Docker - Introducción al Software Libre</p>
</body>
</html>
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker images
                                                                    I Info →  U In Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest       c567b231f656       25.9kB         9.49kB    U
nginx:latest             5ab5b0b01097        243MB         66.8MB    U
servidor-grupo2:latest   c1a52a6bea7d        241MB         63.9MB    U
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS          PORTS
               NAMES
3e28a57b8303   servidor-grupo2   "/docker-entrypoint.…"   48 seconds ago   Up 47 seconds   0.0.0.0:8081->80/tcp, [::]:
8081->80/tcp   servidor-dockerfile
adminsys@servidor-grupo2:/proyecto/docker$
                                                                                        Mostrar el escritorio
```

3.2.2 Red personalizada red-proyecto

Por defecto, los contenedores se conectan a la red bridge de Docker, que no ofrece resolución de nombres entre ellos. Una red puente definida por el usuario aísla el tráfico del proyecto y permite que los contenedores se encuentren por nombre mediante el DNS interno de Docker.

None

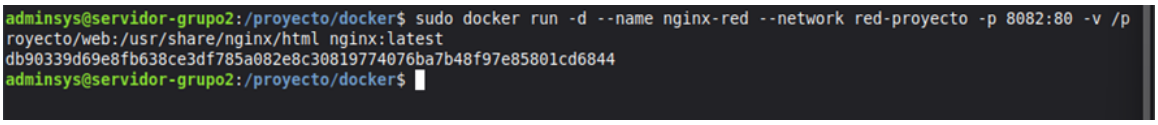
```
sudo docker network create red-proyecto
```

```
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker network create red-proyecto
5c2b9901b65b8cf96012507541eaef510c91e8f7e0a063e18e492dcc20c13387
adminsys@servidor-grupo2:/proyecto/docker$
```

Luego se desplegó un contenedor Nginx llamado nginx-red conectado a esta red (--network red-proyecto). Se expuso en el puerto 8082 y se le montó el directorio /proyecto/web como volumen de tipo bind mount, de modo que sirve el mismo index.html del proyecto.

None

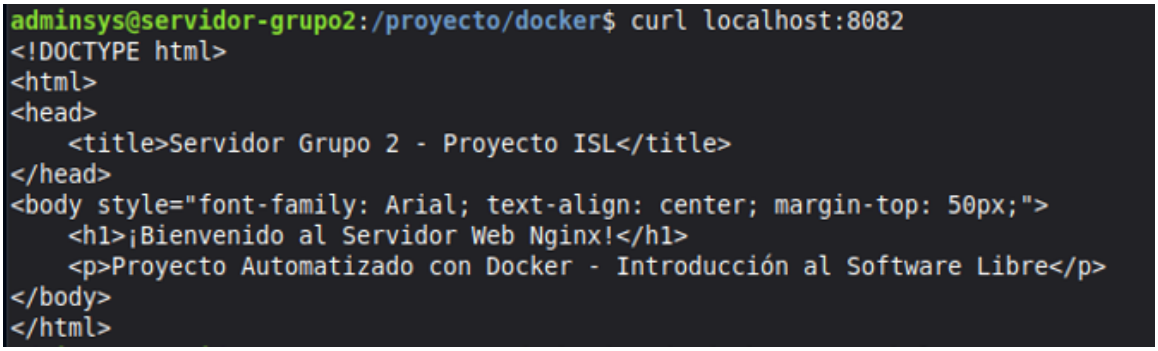
```
sudo docker run -d --name nginx-red --network red-proyecto -p 8084:80 -v /proyecto/web:/usr/share/nginx/html nginx:latest
```



Comprobaciones. Se verificó la respuesta del servicio, se listaron las redes existentes, donde aparece red-proyecto junto a bridge, host y none, y se inspeccionó la red para confirmar que nginx-red está conectado a ella, con su dirección IP asignada.

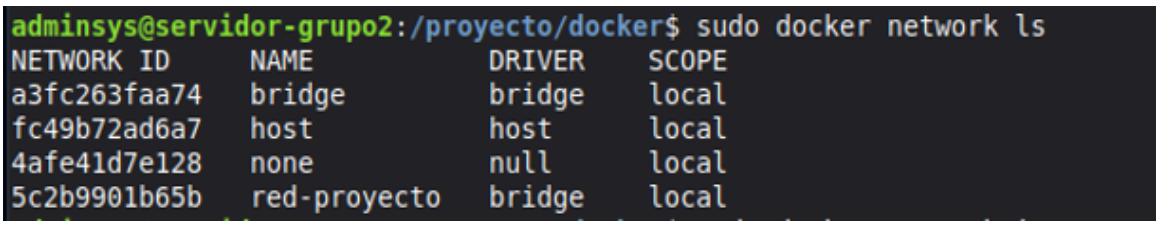
None

```
curl localhost:8082
```



None

```
sudo docker network ls
```



None

```
sudo docker network inspect red-proyecto
```



```
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker network inspect red-proyecto
[
    {
        "Name": "red-proyecto",
        "Id": "5c2b9901b65b8cf96012507541eaef510c91e8f7e0a063e18e492dcc20c13387",
        "Created": "2026-11-03T23:11:19.77686427-06:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {
            "db90339d69e8fb638ce3df785a082e8c30819774076ba7b48f97e85801cd6844": {
                "Name": "nginx-red",
                "EndpointID": "b7b90041fec6780308795fdb50cd46d80efd18dfbc49ed5acd201d6fe19f2e87",
                "MacAddress": "ba:e8:8c:c3:a6:50",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 4,
                        "DynamicIPsAvailable": 65532
                    }
                }
            }
        }
    }
]
```

3.2.3. Volumen persistente volumen-web

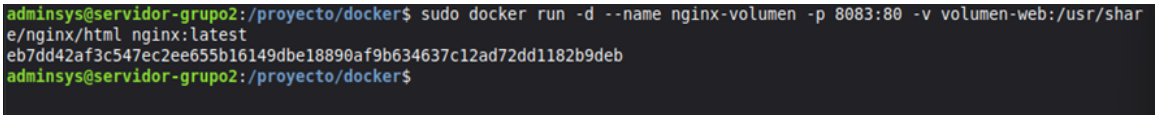
Los datos dentro de un contenedor desaparecen cuando este se elimina. Un volumen nativo de Docker se gestiona fuera del ciclo de vida del contenedor, por lo que la información sobrevive a reinicios y recreaciones.

Primero se creó el volumen:

```
None
sudo docker volume create volumen-web
```

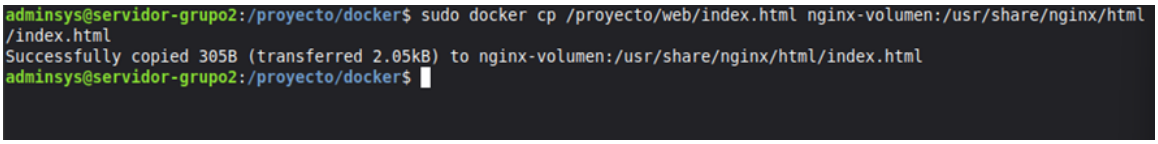
Después se levantó el contenedor nginx-volumen en el puerto 8083, montando el volumen en la ruta donde Nginx publica sus archivos:

```
None
sudo docker run -d --name nginx-volumen -p 8083:80 -v
volumen-web:/usr/share/nginx/html nginx:latest
```



Un volumen nuevo se inicializa con el contenido original de la imagen, es decir, la página de bienvenida por defecto de Nginx. Por eso se copió la página personalizada directamente al volumen:

```
None
sudo docker cp /proyecto/web/index.html
nginx-volumen:/usr/share/nginx/html/index.html
```



Comprobaciones. Se visualizó la página servida desde el volumen, se listó el volumen creado y se inspeccionó para conocer su punto de montaje y su controlador.

None

```
curl localhost:8083
```

```
adminsys@servidor-grupo2:/proyecto/docker$ curl localhost:8083
<!DOCTYPE html>
<html>
<head>
    <title>Servidor Grupo 2 - Proyecto ISL</title>
</head>
<body style="font-family: Arial; text-align: center; margin-top: 50px;">
    <h1>¡Bienvenido al Servidor Web Nginx!</h1>
    <p>Proyecto Automatizado con Docker - Introducción al Software Libre</p>
</body>
</html>
```

None

```
sudo docker volume ls
```

```
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker volume ls
DRIVER    VOLUME NAME
local     volumen-web
```

None

```
sudo docker volume inspect volumen-web
```

```
adminsys@servidor-grupo2:/proyecto/docker$ sudo docker volume ls
DRIVER    VOLUME NAME
local     volumen-web
```

Prueba de persistencia. Para demostrar que los datos no dependen del contenedor, se eliminó nginx-volumen de forma forzada y se creó uno nuevo con el mismo volumen. Aunque el identificador del contenedor cambió, la página personalizada siguió disponible, lo que confirma que la información reside en el volumen.

None

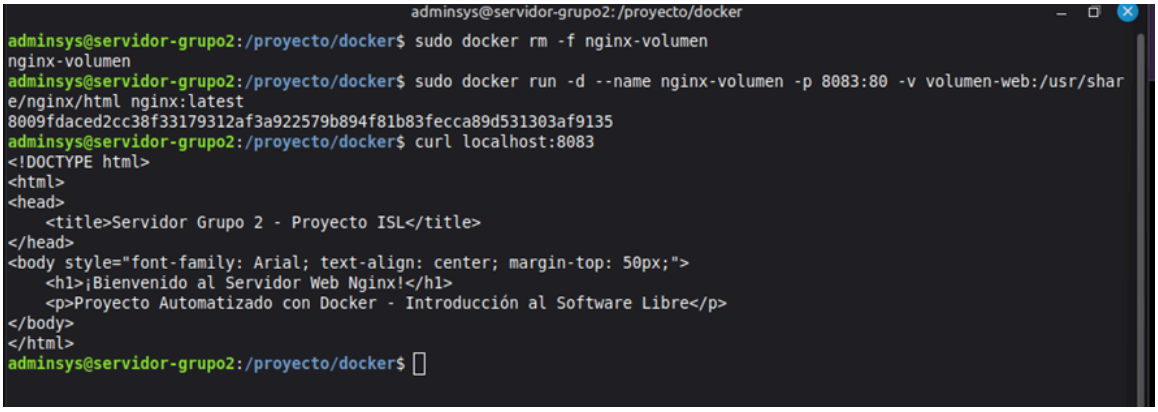
```
sudo docker rm -f nginx-volumen
```

None

```
sudo docker run -d --name nginx-volumen -p 8085:80 -v
volumen-web:/usr/share/nginx/html nginx:latest
```

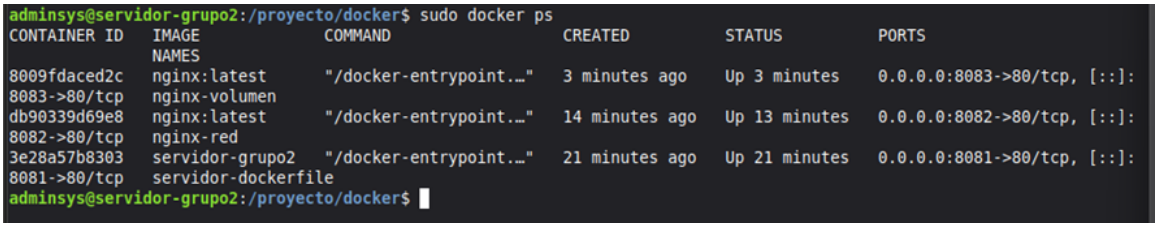
None

```
curl localhost:8085
```



None

```
sudo docker ps
```



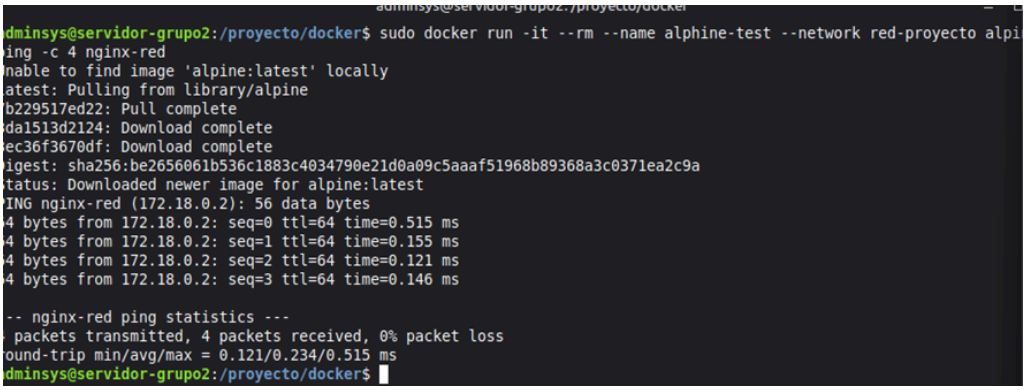
3.3. Pruebas finales y verificación

En esta fase se comprobó que todos los componentes funcionan de forma integrada.

Conectividad interna entre contenedores. Se ejecutó un contenedor temporal de Alpine Linux dentro de red-proyecto (--rm lo elimina al terminar). Con ping hacia nginx-red se comprobó la resolución DNS por nombre de contenedor y la comunicación dentro de la red.

None

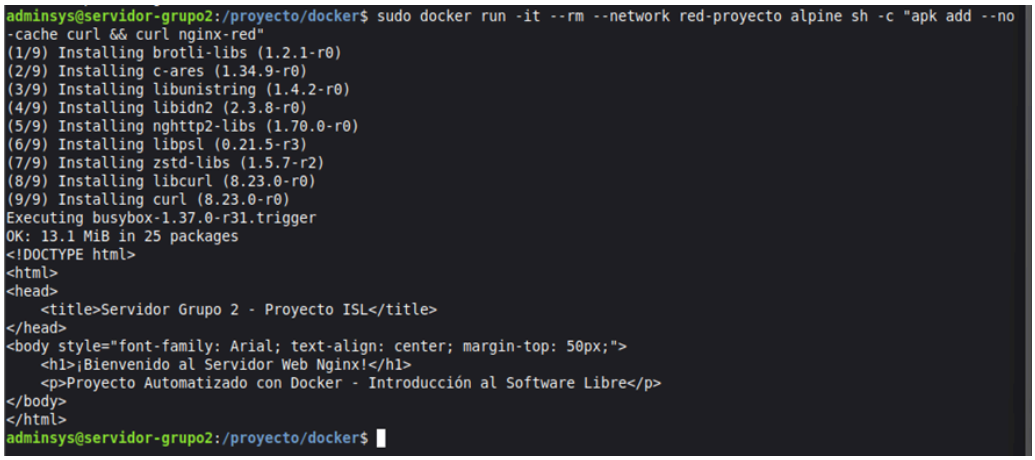
```
sudo docker run -it --rm --name alpine-test --network
red-proyecto alpine ping -c 4 nginx-red
```



Además, se instaló curl en un Alpine temporal y se solicitó la página a nginx-red por su nombre, demostrando que el servicio web es accesible desde otro contenedor de la misma red.

None

```
sudo docker run -it --rm --network red-proyecto alpine sh -c
"apk add --no-cache curl && curl nginx-red"
```



Acceso a los tres servicios. Se consultó cada puerto expuesto al anfitrión:

None

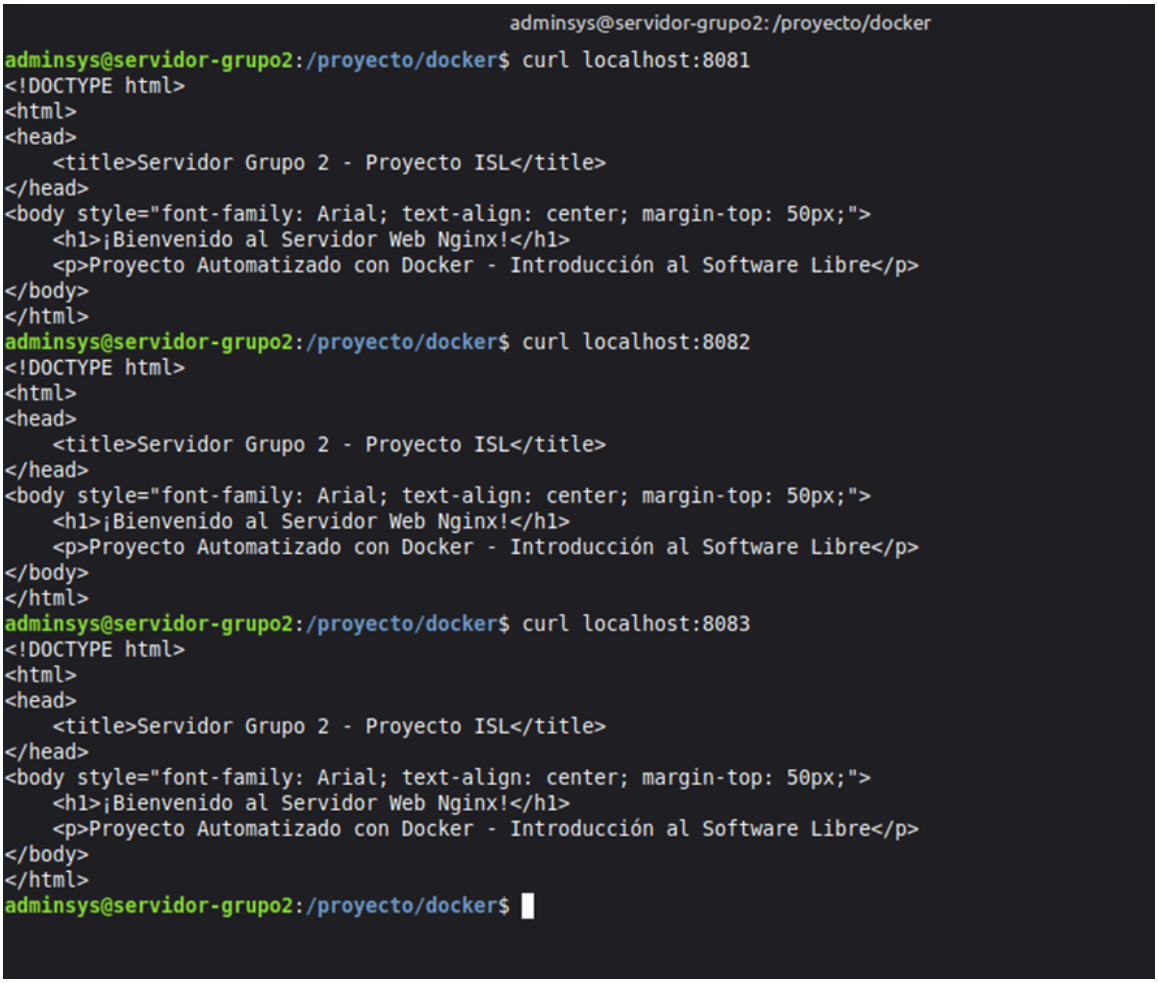
```
curl localhost:8081
```

None

```
curl localhost:8082
```

None

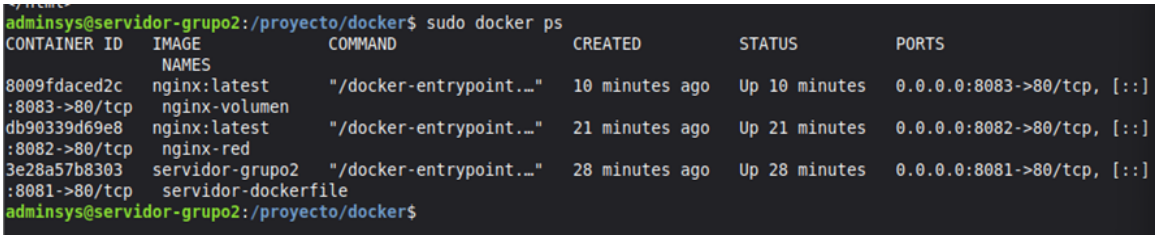
```
curl localhost:8083
```



Estado general de la infraestructura. Se listaron los contenedores activos, las redes y los volúmenes del proyecto.

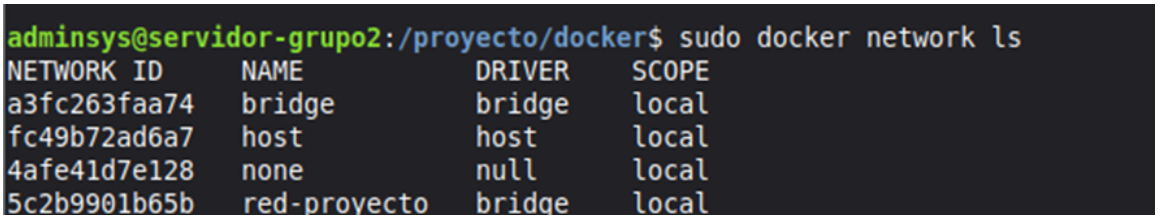
None

```
sudo docker ps
```



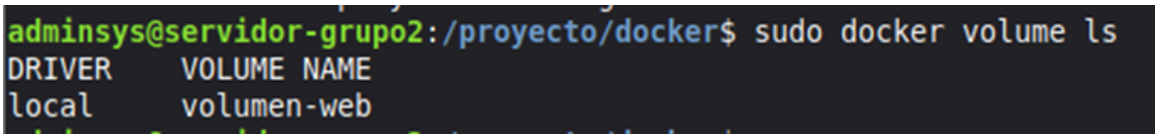
None

```
sudo docker network ls
```



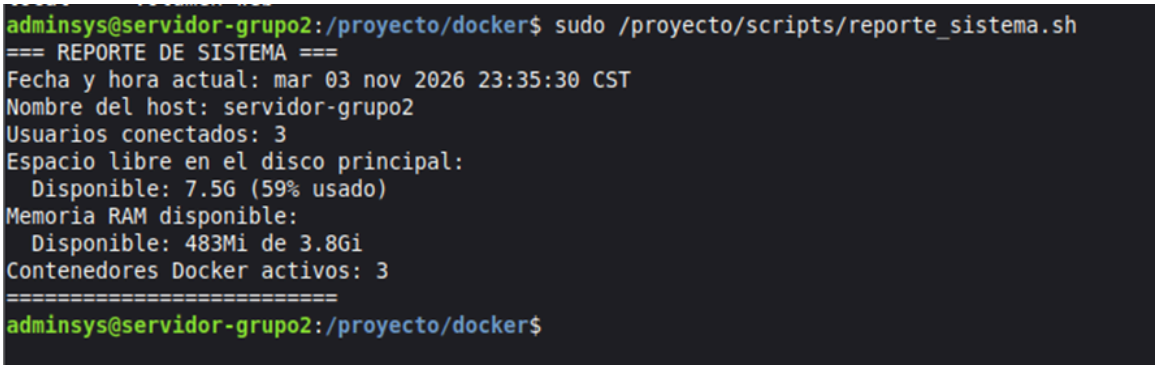
None

```
sudo docker volume ls
```



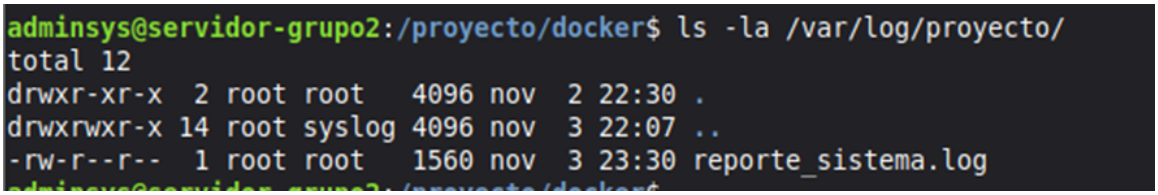
Verificación del monitoreo automatizado. Se ejecutó manualmente el script reporte_sistema.sh para validar su salida, que incluye fecha, host, usuarios, disco, memoria y contenedores activos.

```
None
sudo /proyecto/scripts/reporte_sistema.sh
```

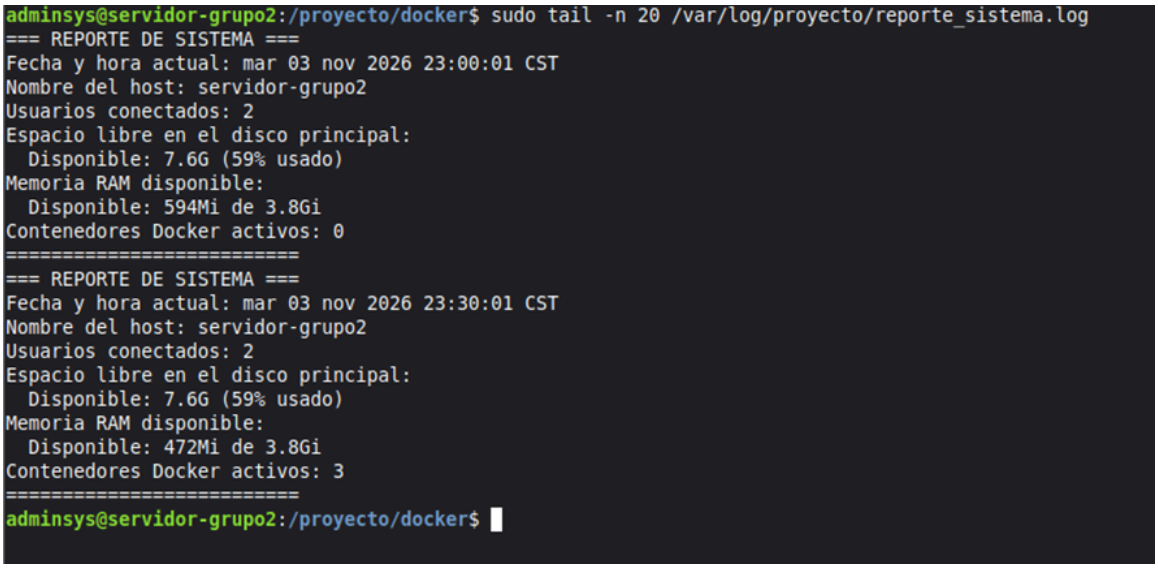


Luego se revisó el directorio de registros y las últimas entradas del log generadas por `cron`:

```
None
ls -la /var/log/proyecto/
```



```
None
sudo tail -n 20 /var/log/proyecto/reporte_sistema.log
```



El log muestra ejecuciones automáticas separadas por 30 minutos (23:00 y 23:30). En la primera el recuento de contenedores activos era 0 y en la segunda era 3, lo que refleja el despliegue de los tres servicios (puertos 8081, 8082 y 8083) entre ambas mediciones y demuestra que el monitoreo detecta los cambios del sistema.

3.3.1 Definición del entorno en una EC2

Para simular un entorno de producción real y altamente disponible, la infraestructura local fue clonada y aprovisionada en la nube pública utilizando Amazon Web Services (AWS). Se levantó una instancia EC2 ejecutando Amazon Linux 2023. El acceso administrativo remoto se aseguró estrictamente mediante el protocolo SSH, requiriendo el intercambio de llaves criptográficas RSA (.pem) y deshabilitando el acceso por contraseña.

En la nube, el despliegue evolucionó. Se abandonaron las ejecuciones manuales aisladas a favor de la construcción de una imagen completamente personalizada definida a través de un 'Dockerfile'. Para garantizar el aislamiento del tráfico, se crearon redes puente (bridge networks) bajo el nombre 'red-proyecto', permitiendo la comunicación interna segura por resolución DNS. La persistencia de la información ante reinicios o destrucción de contenedores se aseguró acoplando volúmenes Docker

nativos ('volumen-web'). Finalmente, los servicios web fueron expuestos al tráfico externo a través de los puertos 8081, 8082 y 8083.

4. Conclusiones y Resultados Obtenidos

El desarrollo de este proyecto demostró de manera práctica la viabilidad y eficiencia de combinar la administración tradicional de sistemas Linux con tecnologías modernas de virtualización a nivel de sistema operativo (Docker). El equipo logró cumplir satisfactoriamente con la rúbrica de evaluación, destacando en los siguientes puntos:

- • Implementación estricta de políticas de seguridad mediante la separación de roles, grupos y el uso de permisos SGID.
- • Reducción de la carga administrativa mediante la creación de scripts de recolección de métricas programados con Cron.
- • Estandarización del entorno de ejecución al migrar los servicios de red hacia contenedores portables (Nginx).
- • Escalabilidad demostrada al llevar la arquitectura desde una máquina local hacia un entorno Cloud computing (AWS EC2).

El uso riguroso de Git como herramienta de control de versiones fue el pilar que permitió coordinar los esfuerzos individuales en un entorno asíncrono, resultando en un sistema robusto, documentado y listo para producción.